QUERYQUEST AI

AI Database Assistant

Project report and artifact guide

React + Vite · Vercel-ready · OpenAI Responses API

October 30, 2026

QueryQuest AI is an educational web application for learning SQL through natural-language questions, explanations, and safe practice using fictional sample data.

1 Project overview

QueryQuest AI helps a learner move from a question in everyday language to a SQL draft they can inspect and learn from. The interface combines an AI prompt, editable SQL, a plain-language explanation, a schema browser, a practice challenge, and a small results preview. The implementation is a frontend application with a serverless AI endpoint prepared for deployment.

1.1 Goals

- Make SQL more approachable by starting with a natural-language question.
- Show generated queries with explanations so learners can review the logic.
- Offer a fictional ShopSmart schema for practice without real customer data.
- Keep the AI credential server-side and configure the project for Vercel.

2 Application design

The app uses React and Vite. React state controls the prompt, SQL editor, explanation, challenge, errors, and results preview. Vite supplies local development and production frontend builds. The layout adapts to narrow screens and includes workspace navigation, schema cards, query controls, and a daily challenge. The sample schema has four fictional tables: `products`, `customers`, `orders`, and `order_items`. Its relationships are provided to the AI to inform common joins.

2.1 AI request flow

1. The browser sends the question to the same-origin `/api/generate` endpoint.
2. The serverless function validates the request length and checks the API key configuration.
3. The function sends the question and schema to the OpenAI Responses API, requesting structured JSON containing SQL, an explanation, and a practice prompt.
4. The browser displays the draft for review. The endpoint does not execute the SQL or connect to a database.

The API key is read from the deployment environment and is not embedded in the frontend. The model is configurable through `OPENAI_MODEL`; the default is `gpt-4.1-mini`.

2.2 Safe practice preview

The browser preview uses a small set of fictional rows. It supports basic single-table `SELECT`, `WHERE`, `ORDER BY`, and `LIMIT` examples. It is an instructional preview, not a PostgreSQL engine. Complex joins and aggregate SQL may be generated for learning but are not fully evaluated. Common data-changing keywords are rejected. This preview is not a security boundary and must not connect directly to a production database.

3 Project artifacts

`src/main.jsx`	React app, sample schema, AI request interaction, and query preview.
`src/style.css`	Responsive visual styling.
`api/generate.js`	Vercel function for AI SQL generation and explanation.

`vercel.json`	Vercel routing configuration.
`package.json`	React/Vite dependencies and development/build commands.
`.env.example`	Server-side environment variable names; no secrets.
`README.md`	Local setup, deployment steps, scope, and safety notes.

4 Run and deployment

For local frontend development, install Node dependencies and run `npm run dev`. To test the serverless API locally, use Vercel CLI with a local environment file containing `OPENAI_API_KEY`; keep it out of version control. To deploy, import the project directory into Vercel, set `OPENAI_API_KEY` in project environment variables, optionally set `OPENAI_MODEL`, then deploy. Vercel serves the frontend and `api/generate.js` function. A key is required for AI generation; provider usage may incur charges.

5 Links and publication status

The application is publicly deployed on Vercel and its source is published in a public GitHub repository.

- Live app: https://queryquest-ai.vercel.app.
- GitHub repository: github.com/Armstrfa/queryquest-ai.

The site is public. AI generation requires `OPENAI_API_KEY` to be added to the Vercel project's encrypted environment variables; until then the API returns a setup message.

6 Limitations and next steps

AI generation requires an OpenAI API key and has not been exercised against a live provider from this workspace. Generated SQL may be wrong and should be reviewed. The preview runner supports only a restricted SQL subset on fictional rows. The endpoint's in-memory rate cap is best-effort per warm serverless instance, not a durable distributed limiter. The challenge is a learning prompt; persistent scoring and accounts are not implemented. Next, add the key as a protected Vercel environment variable and redeploy. Before adding real database connectivity, implement server-side SQL parsing, authorization, execution limits, and a read-only database role.